



Instituto Superior de Engenharia de Lisboa
Departamento de Engenharia Informática

Desenvolvimento de Aplicações Móveis

Projeto Final - BaseLift

Docente: Pedro Fazenda

Turma 61D

Guilherme Graça - a51827

10 de junho de 2026

Índice

1	Relatório do Projeto BaseLift	1
1.1	Descrição Geral	1
1.2	Motivação e Contexto	1
1.3	Modelo de Negócio	1
1.4	Arquitetura	1
1.4.1	Injeção de Dependências	2
1.5	Gestão de Estado MVVM com StateFlow	2
1.6	Stack Tecnológica	3
1.7	Base de Dados (Room)	3
1.7.1	Entidades (Tabelas)	3
1.7.2	Relações entre Tabelas	3
1.7.3	Migrações	4
1.7.4	DAOs	4
1.8	Funcionalidades Implementadas	4
1.8.1	OnBoarding e Perfil	4
1.8.2	Cálculo de Métricas (CalculatorUtils)	4
1.8.3	Nutrição	5
1.8.4	Treinos	5
1.8.5	Progresso de Peso e Diário Visual (Insights)	6
1.8.6	Dashboard	7
1.9	Componentes UI Reutilizáveis	8
1.9.1	InteractiveChartWithControls	8
1.9.2	BarChartWithControls	8
1.9.3	ChartDataPoint	8
1.9.4	CustomCalendar	8
1.10	Navegação	9
1.11	Design e Experiência de Utilizador (UX)	9
1.12	Processo de Desenvolvimento	10
1.13	Otimizações de Performance	10
1.14	Estrutura de Ficheiros (Resumo)	11
1.15	Estado Atual do Desenvolvimento	12
1.16	O que Falta / Possíveis Próximos Passos	13

1. Relatório do Projeto BaseLift

1.1 Descrição Geral

A **BaseLift** é uma aplicação Android de fitness e nutrição desenvolvida em Kotlin. O objetivo é centralizar num só lugar o rastreio de treinos, nutrição e peso corporal, sem as distrações comuns de apps semelhantes (tutoriais, publicidade, funcionalidades bloqueadas por subscrição).

A app é direcionada para utilizadores que já treinam, por isso a interface apresenta apenas os dados essenciais. Funciona 100% offline todos os dados são guardados localmente no dispositivo.

1.2 Motivação e Contexto

O projeto surge de uma necessidade pessoal. As funcionalidades típicas de rastreio fitness estão espalhadas por múltiplas apps, e muitas delas bloqueiam as funções mais úteis atrás de subscrições pagas. A ideia foi construir uma alternativa gratuita, offline-first, com uma interface de alta qualidade focada em quem já sabe o que está a fazer na academia.

1.3 Modelo de Negócio

O modelo é faseado:

- **Fase 1 (atual):** todas as funcionalidades são gratuitas para maximizar a base de utilizadores.
- **Fase 2 (futura):** introdução de funcionalidades avançadas (base de dados global de alimentos, estimativa de calorias por IA a partir de fotos) num modelo de subscrição mensal. Estas dependem de APIs externas pagas. As funcionalidades base continuariam gratuitas e offline.

1.4 Arquitetura

A app segue o padrão **MVVM** (Model-View-ViewModel), que é a abordagem recomendada pela Google para Android moderno.

Model

```
local/  
    entity/      -> data classes anotadas com @Entity (tabelas Room)  
    dao/         -> interfaces com queries SQL (@Dao)  
    AppDatabase -> configuração singleton do Room  
repository/     -> camada de abstração entre DAOs e ViewModels
```

ViewModel

```
onboarding/     -> OnboardingViewModel  
nutrition/      -> NutritionViewModel  
workout/        -> WorkoutViewModel  
progress/       -> ProgressViewModel
```

dashboard/ -> DashboardViewModel

View (Jetpack Compose)

onboarding/ -> OnboardingScreen, CustomTargetsScreen
nutrition/ -> NutritionScreen + componentes
workout/ -> WorkoutScreen + componentes
insights/ -> InsightsScreen + componentes
dashboard/ -> DashboardScreen + componentes
navigation/ -> AppNavigation (grafo de navegação)
components/ -> componentes reutilizáveis de gráficos
theme/ -> tema visual (cores, tipografia, dark mode)

1.4.1 Injeção de Dependências

É usada **injeção manual** sem Hilt ou Dagger. O `AppContainer` é uma interface que define todos os repositórios disponíveis. A implementação `DefaultAppContainer` instancia-os de forma lazy (só quando necessários). É inicializado uma vez na `BaseLiftApplication`.

```
1 interface AppContainer {  
2     val database: AppDatabase  
3     val userRepository: UserRepository  
4     val progressRepository: ProgressRepository  
5     val workoutRepository: WorkoutRepository  
6     val nutritionRepository: NutritionRepository  
7 }
```

1.5 Gestão de Estado MVVM com StateFlow

O fluxo de dados segue sempre a mesma direção:

Room DB -> Flow<T> (DAO) -> Repository -> StateFlow<T> (ViewModel) -> Composable

O Room emite um Flow diretamente a partir de cada query quando a base de dados muda, a UI atualiza-se automaticamente sem polling ou callbacks manuais.

Cada ViewModel expõe o estado como `StateFlow` imutável (via `.asStateFlow()`). O `MutableStateFlow` interno só pode ser alterado pelo próprio ViewModel; a View apenas lê e chama funções públicas. A conversão de Flow para StateFlow é feita com `.stateIn(scope = viewModelScope, started = SharingStarted.Lazily)` o Lazily garante que o flow só corre quando há observadores ativos.

Quando o estado depende de múltiplas fontes (ex: utilizador + logs + templates), usa-se `combine()`. O `DashboardViewModel` combina 7 flows em simultâneo. Os cálculos pesados (streaks, volumes, tendências) correm em `Dispatchers.Default` via `flowOn()` para não bloquear a Main Thread.

Nem todo o estado vai para o ViewModel. Estado puramente de UI (campo expandido, diálogo aberto, posição de swipe) fica em `remember {}` no Composable é descartado ao sair do ecrã. Estado que precisa de persistir durante a sessão (sessão ativa, rascunhos de peso/reps) fica no ViewModel em `mutableStateMapOf`.

Na View, os `StateFlow` são observados com `collectAsStateWithLifecycle()`, que para de coletar quando a app vai para segundo plano, evitando processamento desnecessário.

1.6 Stack Tecnológica

Componente	Tecnologia
Linguagem	Kotlin
UI Framework	Jetpack Compose + Material 3
Navegação	Navigation Compose
Base de Dados	Room Database (SQLite)
Async / Reatividade	Kotlin Coroutines + StateFlow / Flow
Gráfico de Peso	Android Canvas (personalizado)
Outros Gráficos	Biblioteca Vico
Carregamento de Imagens	Coil (<code>coil-compose:2.5.0</code>)
Processamento Código Room	KSP (Kotlin Symbol Processing)
<code>compileSdk</code>	35 (Android 15)
<code>minSdk</code>	24 (Android 7.0)

1.7 Base de Dados (Room)

A base de dados chama-se `baselift_database` e está na versão **10**. O Room gera a implementação automaticamente via KSP.

1.7.1 Entidades (Tabelas)

Entidade	Tabela	Descrição
UserEntity	<code>user_table</code>	Perfil único do utilizador (<code>id=1</code>)
WeightLogEntity	<code>weight_logs</code>	Histórico de registos de peso
PhotoLogEntity	<code>photo_logs</code>	URIs de fotos do diário visual
WorkoutEntity	<code>workouts</code>	Planos/rotinas de treino
ExerciseEntity	<code>exercises</code>	Exercícios dentro de um workout
WorkoutSessionEntity	<code>workout_sessions</code>	Sessões de treino (ativas/concluídas)
SetLogEntity	<code>set_logs</code>	Registo de cada série (peso, reps, PR)
NutritionLogEntity	<code>nutrition_logs</code>	Registo diário de calorias/macros
MealTemplateEntity	<code>meal_templates</code>	Templates de refeições pré-configuradas

1.7.2 Relações entre Tabelas

- ExerciseEntity tem chave estrangeira para WorkoutEntity (CASCADE DELETE)
- SetLogEntity tem chaves estrangeiras para WorkoutSessionEntity e ExerciseEntity (CASCADE DELETE)
- WorkoutSessionEntity fica ligada ao WorkoutEntity por `workoutId`

1.7.3 Migrações

O esquema evoluiu ao longo do desenvolvimento com migrações incrementais:

- **v7→v8**: coluna `setCount` adicionada à tabela `exercises`
- **v8→v9**: criação das tabelas `nutrition_logs` e `meal_templates`
- **v9→v10**: criação de índices em `timestamp`, `isCompleted`, para melhorar a performance das queries

1.7.4 DAOs

Cada DAO define as queries SQL para a sua tabela. As queries que devolvem dados reactivos usam `Flow<...>` para atualizar a UI automaticamente. As queries de escrita são `suspend fun`.

Exemplos de queries relevantes no `WorkoutDao`:

```
1  -- peso máximo de um exercício (para detetar PR de peso)
2  SELECT MAX(sl.weight) FROM set_logs sl
3  INNER JOIN workout_sessions ws ON sl.sessionId = ws.id
4  WHERE sl.exerciseId = :exerciseId AND ws.isCompleted = 1 AND sl.isCompleted = 1
5
6  -- 1RM máximo estimado (para detetar PR de volume)
7  SELECT MAX(sl.weight * (1.0 + (sl.reps / 30.0))) FROM set_logs sl
8  INNER JOIN workout_sessions ws ON sl.sessionId = ws.id
9  WHERE sl.exerciseId = :exerciseId AND ws.isCompleted = 1 AND sl.isCompleted = 1
```

1.8 Funcionalidades Implementadas

1.8.1 OnBoarding e Perfil

O fluxo de entrada recolhe os dados do utilizador em 4 passos:

- **Dados biométricos**: género, idade, peso (KG ou LBS), altura (CM ou FT)
- **Nível de atividade**: Sedentary → Extra Active
- **Objetivo**: Extreme Loss → Extreme Gain (7 opções)
- **Resumo calculado**: calorias e macros diários calculados dinamicamente

O utilizador pode também definir **metas personalizadas** (Custom Targets) em vez de usar os valores calculados.

Recalibração (Smart Recalibration): em qualquer altura, o utilizador pode recalibrar a baseline na área de Insights. O fluxo de onboarding é reutilizado para este efeito o botão final muda para "UPDATE BASELINE" e no final redireciona de volta para os Insights.

1.8.2 Cálculo de Métricas (CalculatorUtils)

Toda a lógica de cálculo está isolada no objeto `CalculatorUtils`:

BMR (Mifflin-St Jeor):

- Masculino: $(10 \times \text{kg}) + (6.25 \times \text{cm}) - (5 \times \text{idade}) + 5$

- Feminino: $(10 \times \text{kg}) + (6.25 \times \text{cm}) - (5 \times \text{idade}) - 161$

TDEE: $\text{BMR} \times \text{multiplicador de atividade (1.2 a 2.1)}$

Ajuste calórico por objetivo: de -1000 kcal (Extreme Loss) a +1000 kcal (Extreme Gain)

Mínimo de segurança: o resultado é sempre limitado a ≥ 1200 kcal

Divisão de macros:

- Proteína: $30\% \text{ das calorias} \div 4 \text{ kcal/g}$
- Hidratos: $45\% \text{ das calorias} \div 4 \text{ kcal/g}$
- Gordura: $25\% \text{ das calorias} \div 9 \text{ kcal/g}$

BMI: $\text{peso(kg)} / \text{altura(m)}^2$

Quando o utilizador tem Custom Targets, só o BMI é recalculado os macros permanecem como definidos.

1.8.3 Nutrição

O ecrã de nutrição é composto por 5 componentes independentes:

DailyProgressSection: gráfico circular das calorias restantes + barras lineares de Proteína, Hidratos e Gordura. Actualiza em tempo real conforme os registos do dia.

QuickLogSection: registo rápido com dois modos:

- *Calories Only:* apenas calorias
- *Detailed Macros:* calorias + proteína + hidratos + gordura

QuickAddMealsSection: templates de refeições pré-configuradas. Toque único para registar. Botão de configuração para criar/editar templates com nome, ícone e macros (as calorias são calculadas automaticamente).

TodaysLogsSection: tabela com os registos do dia. Inclui botão "Reset Logs" para apagar todos. Long-press num registo ativa uma animação a vermelho e abre diálogo de confirmação de eliminação.

Lógica de data: os registos usam timestamps. O `NutritionViewModel` usa um `refreshTrigger` para garantir que os logs do dia correto são carregados (os dados dão reset à meia-noite automaticamente do ponto de vista da UI).

1.8.4 Treinos

O ecrã de treinos é unificado gere rotinas e regista cargas na mesma view.

Estrutura de dados:

- `WorkoutEntity`: nome do plano (ex: "Push Day")
- `ExerciseEntity`: exercício dentro do plano (nome, equipamento, grupos musculares, nº séries)
- `WorkoutSessionEntity`: sessão ativa ou concluída
- `SetLogEntity`: cada série com peso, reps e tipo de PR

Estado da UI (`WorkoutUiState`):

- Lista de workouts disponíveis
- Workout selecionado
- Sessão ativa
- Lista de exercícios com as séries (`ExerciseUiModel` $\rightarrow$ `List<SetUiModel>`)

Tipos de série: Normal, Warm-up, Drop-set, Failure (com visual diferenciado)

Lógica de séries:

- Cada exercício tem um `setCount` que define o número de séries
- As séries têm estado sequencial: só se pode completar a próxima se a anterior estiver concluída
- É possível adicionar/remover séries durante a sessão
- Os rascunhos de peso/reps são mantidos em `mutableStateMapOf` (sobrevivem a recomposições)

Personal Records (PRs):

- **PR de Peso:** o peso desta série supera o máximo histórico do exercício
- **PR de Volume (1RM estimado):** $\text{peso} \times (1 + \text{reps}/30)$ supera o 1RM histórico
- Detecção automática ao completar uma série, guardada no campo `prType` do `SetLogEntity`

Rest Timer: temporizador de descanso integrado com controlos rápidos (+30s, -10s, Skip)

Plate Calculator: calculadora de discos integrada no campo de peso dado um peso alvo e o peso da barra, indica quais discos colocar em cada lado

Drag-and-drop: reordenação de exercícios

Sessões: quando o utilizador inicia um treino, é criada uma `WorkoutSessionEntity` com `isCompleted` = `false`. Ao terminar o treino, a sessão fica marcada como completa e os rascunhos são limpos.

Cache de queries: o `WorkoutViewModel` mantém um cache local `previousSetsCache` para evitar N+1 queries à base de dados ao reconstruir o estado da UI.

1.8.5 Progresso de Peso e Diário Visual (Insights)

Gráfico de Peso (Canvas personalizado):

- Desenhado inteiramente com `android.graphics.Canvas` sem bibliotecas externas
- Cor Verde Néon (`#CCFF00`)
- Eixo Y dinâmico: começa no menor peso registado (não em zero), para realçar variações subtis
- Linha de meta tracejada em Azul Elétrico (`#00E5FF`) quando definida
- Tooltips interativos ao tocar num ponto (peso + data)
- Exibe data de início, média e data de fim no topo; valor médio no eixo Y
- Filtros temporais: 7D, 30D, 1Y, ALL

Technical History Ledger:

- Lista os registos de peso mais recentes
- Inicialmente mostra 4 registos com fade-out no último
- Botão "LOAD MORE DATA" para expandir / "LOAD LESS DATA" para colapsar
- Long-press: animação de preenchimento gradual vermelho de 0% a 100% → abre diálogo de confirmação glassmorphic

Delta inteligente: o indicador de diferença (+/- KG) usa Verde Néon ou Coral Suave consoante a direção é favorável ou desfavorável ao objetivo do utilizador (perder vs ganhar peso). Recalculado automaticamente em recalibrações.

Atualização automática de peso: ao inserir um registo com data de hoje, o `weight` do `UserEntity` é atualizado e o BMI é recalculado.

Validação de datas: datas futuras são bloqueadas tanto no picker de peso como no de fotos.

Diário Visual:

- Fotos selecionadas da galeria nativa via `ActivityResultContracts.PickVisualMedia`
- Persistência offline: `takePersistableUriPermission` garante acesso à URI após reinício
- Seleção em lote (batch upload): múltiplas fotos com a mesma data
- Ordenação cronológica da esquerda (mais antiga) para a direita (mais recente, marcada como LATEST)
- Gradientes de scroll dinâmicos nas extremidades
- Detalhe ao tocar: zoom gestual (pinch-to-zoom), arrasto, duplo toque para reset, cruzamento automático com o peso registado nesse dia

1.8.6 Dashboard

O Dashboard agrega dados de nutrição e treino para dar uma visão geral da consistência do utilizador.

Consistency Journal (streaks):

- **Nutrition Streak:** conta dias com registo de nutrição em semanas consecutivas que cumpriram a meta (configurável: 0-7 dias de descanso por semana)
- **Workout Streak:** conta dias com treino em semanas consecutivas que cumpriram a meta
- A semana atual nunca quebra a streak (mesmo que incompleta)
- Cartões com animação de glow quando a streak atinge 7 dias
- **Calendário semanal:** mostra os 7 dias da semana (SegDom) com marcadores visuais de nutrição e treino

Total Training Volume:

- Gráfico de barras (`BarChartWithControls`) com volume total semanal (SUM(peso × reps) por semana ISO)

- Filtros temporais: 1M, 3M, 1Y
- Mostra valor total no período e variação percentual face ao período anterior

Routines (menus colapsáveis de dois níveis):

- **Nível 1:** lista de workouts. Ao expandir, gráfico de volume desse workout ao longo do tempo
- **Nível 2:** lista de exercícios dentro do workout. Ao expandir:
 - Cartão de PR de Peso Máximo absoluto
 - Cartão de PR de Volume Máximo (peso $\times$ reps)
 - Gráfico de Volume ao longo do tempo (verde néon)
 - Gráfico de Peso Máximo ao longo do tempo (azul elétrico)
- Carregamento lazy: os dados de cada exercício só são carregados quando o submenu é expandido

Calendário histórico: registo global de todos os dias com nutrição e/ou treino, disponível no componente `CustomCalendar`.

1.9 Componentes UI Reutilizáveis

1.9.1 `InteractiveChartWithControls`

Gráfico de linhas/área com:

- Filtros temporais (7D, 30D, 1Y, ALL) ou escala cronológica (toggle)
- Tooltips interativos (suporte a `\n` para múltiplas linhas)
- Botão para definir/remover objetivo
- Cor parametrizável
- Desenhado via Canvas; 24 KB de código

1.9.2 `BarChartWithControls`

Gráfico de barras com:

- Filtros temporais (1M, 3M, 1Y)
- Resumo de volume total + variação percentual

1.9.3 `ChartDataPoint`

Modelo de dados genérico: `xValue: Long, yValue: Float, tooltipLabel: String, extraValue: Float`

1.9.4 `CustomCalendar`

Calendário navegável mês a mês com marcadores de dias de nutrição e treino. Permite ver histórico de toda a atividade.

1.10 Navegação

A navegação é gerida por `AppNavigation.kt` com Navigation Compose. O grafo inclui:

- `onboarding` → `custom_targets` → `dashboard`
- `dashboard` (tab) ↔ `nutrition` (tab) ↔ `workout` (tab) ↔ `insights` (tab)

Existe um `TopAppBar` global com foto de perfil e ícone de definições (acesso a Reset total de dados).

A barra de navegação inferior (`BottomNavigationBar`) tem 4 tabs: Dashboard, Nutrition, Workout, Insights.

1.11 Design e Experiência de Utilizador (UX)

O design foi um dos focos principais do projeto. Paleta exclusivamente dark mode, tipografia em maiúsculas, e nenhum elemento decorativo cada pixel tem um propósito.

Paleta de cores:

Cor	Hex	Uso
Fundo	#000000 / #131313	Ecrãs e cards
Verde Néon	#CCFF00	CTA, progresso positivo, gráfico de peso
Azul Elétrico	#00E5FF	Linha de meta, hidratos, gráfico de peso máximo
Coral Suave		Alertas, ações destrutivas, delta desfavorável

Os diálogos e overlays usam glassmorphism: gradiente linear subtil (`alpha 0.08` → `0.02`) + borda `alpha 0.15`.

Padrões de eliminação:

- *Registos simples* (nutrição e peso): **long-press** durante 600ms anima um gradiente Coral da esquerda para a direita, depois abre diálogo de confirmação.
- *Templates Quick Add*: como o toque simples já regista a refeição, usar long-press causaria eliminações acidentais. Foi implementado **swipe-to-delete** para a direita: o item desloca-se (foreground com `offset`), revelando um background vermelho com ícone de lixo que cresce de 50% → 100% consoante o progresso. Ao cruzar 80dp, dispara feedback háptico. Se soltar antes, faz snap de volta com spring animation. Ao confirmar, a saída é encadeada: slide → shrink vertical → fade.
- *Fotos*: fluxo de 2 passos no próprio botão (REMOVE → CANCEL / CONFIRM), sem diálogo extra.

Outros elementos de UX:

- Cards colapsáveis com fade-out no último item visível (`alpha 0.4`), sugerindo que há mais conteúdo
- Gráfico de peso em Canvas com eixo Y dinâmico (começa no mínimo, não no zero), área preenchida, linha de meta tracejada e tooltips por toque

- Cartões de streak com efeito glow (halo radial + sombra com blurRadius 25f) ao atingir ≥ 7 semanas
- Tooltips no calendário semanal ao tocar num dia ativo
- Gradientes de scroll animados nas extremidades do carrossel de fotos

1.12 Processo de Desenvolvimento

Fase de mockups: antes de escrever código, foram criados mockups dos ecrãs principais (onboarding, nutrição, workout, Insights, Dashboard). Nesta fase ficaram definidos a paleta, a tipografia em maiúsculas e o sistema de cards. Ter uma referência visual concreta poupou iterações durante a implementação.

Vertical Slicing: o desenvolvimento foi estruturado em 6 fases. A Fase 1 foi horizontal (fundações: base de dados, tema, DI, navegação). As Fases 2 a 6 foram verticais cada uma entregou uma funcionalidade completa da BD até à UI, utilizável no final da fase:

Fase 1 -> App compila + tema + navegação

Fase 2 -> Onboarding + cálculo de metas

Fase 3 -> Registo de peso + gráfico + diário de fotos

Fase 4 -> Treinos + registo de séries + PRs

Fase 5 -> Nutrição + templates de refeições

Fase 6 -> Dashboard + streaks + gráficos históricos

Isto permitiu usar a app a sério durante o desenvolvimento, o que revelou problemas reais de UX. Por exemplo, depois da Fase 5 detetou-se que os logs não faziam reset correto ao mudar de dia, o que levou à criação do `refreshTrigger` no `NutritionViewModel`.

Refinamentos finais: após todas as fases, houve uma fase de polimento focada em performance e UX, motivada por lentidão na navegação entre tabs. O crossfade da `NavHost` foi removido, foi adicionado Deferred Rendering no Dashboard e Workout (50ms de delay nas listas), e o gráfico de peso foi substituído do Vico por Canvas personalizado para controlo total sobre o aspeto visual.

1.13 Otimizações de Performance

- **Remoção do crossfade** na `NavHost` entre tabs para evitar sobreposição de ecrãs
- **Deferred Rendering** no Dashboard e Workout: os cabeçalhos são renderizados de imediato, as listas pesadas são desenhadas com 50ms de delay para eliminar sensação de bloqueio
- **Índices na base de dados** (migração v9→v10): em `timestamp`, `isCompleted`, etc., para acelerar queries históricas
- **Cache de queries** no `WorkoutViewModel` (`previousSetsCache`) para evitar N+1 queries ao reconstruir o estado dos exercícios
- **`flowOn(Dispatchers.IO / Default)`** em todos os flows pesados para não bloquear a Main Thread
- **`SharingStarted.Lazily`** nos `StateFlow` para não calcular estado quando não há observadores
- **Limpeza de `systemBarsPadding()`** globais para eliminar espaços em branco desnecessários no Dashboard e Insights

1.14 Estrutura de Ficheiros (Resumo)

```
com.example.baselift/  
  AppContainer.kt                <- DI Manual  
  BaseLiftApplication.kt         <- Application class  
  MainActivity.kt                <- Ponto de entrada  
  
  Utils/  
    CalculatorUtils.kt           <- BMR, TDEE, BMI, macros  
  
  Model/  
    local/  
      AppDatabase.kt             <- Singleton Room (v10, 9 entidades)  
      dao/  
        UserDao.kt  
        WeightLogDao.kt  
        PhotoLogDao.kt  
        WorkoutDao.kt            <- queries de PR, sessions, sets  
        NutritionDao.kt  
      entity/  
        UserEntity.kt  
        WeightLogEntity.kt  
        PhotoLogEntity.kt  
        WorkoutEntity.kt  
        ExerciseEntity.kt  
        WorkoutSessionEntity.kt  
        SetLogEntity.kt          <- inclui prType  
        NutritionLogEntity.kt  
        MealTemplateEntity.kt  
      repository/  
        UserRepository.kt  
        ProgressRepository.kt  
        WorkoutRepository.kt     <- lógica de PR e sessions  
        NutritionRepository.kt  
  
  ViewModel/  
    onboarding/OnboardingViewModel.kt  
    nutrition/NutritionViewModel.kt  
    workout/WorkoutViewModel.kt  
    progress/ProgressViewModel.kt  
    dashboard/DashboardViewModel.kt <- 461 linhas, streaks, volumes, tendências  
  
  View/  
    theme/                        <- Dark mode, cores, tipografia
```

```

navigation/AppNavigation.kt  <- Grafo completo + TopHeaderBar
components/
    InteractiveChartWithControls.kt  <- 24KB, Canvas
    BarChartWithControls.kt          <- 15KB
    ChartDataPoint.kt
    CustomCalendar.kt                <- 12KB
onboarding/
    OnboardingScreen.kt
    CustomTargetsScreen.kt
    components/OnboardingComponents.kt
nutrition/
    NutritionScreen.kt
    components/
        DailyProgressSection.kt
        QuickLogSection.kt
        QuickAddMealsSection.kt
        TodaysLogsSection.kt
        ConfigureMealDialog.kt
workout/
    WorkoutScreen.kt
    components/
        ExerciseCard.kt
        SetRow.kt
        WorkoutDialogs.kt
insights/
    InsightsScreen.kt
    components/InsightsComponents.kt  <- 38KB
dashboard/
    DashboardScreen.kt
    components/
        ConsistencyJournalSection.kt  <- 24KB
        RoutinesSection.kt            <- 21KB
        TotalTrainingVolumeSection.kt

```

1.15 Estado Atual do Desenvolvimento

Todas as 6 fases do plano original estão concluídas:

Fase	Descrição	Estado
1	Fundações (dependências, tema, Room, DI, navegação)	✓ Concluído
2	OnBoarding, perfil, Insights baseline	✓ Concluído
3	Progresso de peso e diário visual	✓ Concluído
4	Planeamento e registo de treinos	✓ Concluído
5	Nutrição	✓ Concluído
6	Dashboard com streaks, volumes e rotinas	✓ Concluído

A app está funcional e todas as funcionalidades descritas no plano original foram implementadas, incluindo as otimizações de performance adicionadas após a fase 6.

1.16 O que Falta / Possíveis Próximos Passos

- **Testes unitários:** o plano previa JUnit para `OnboardingViewModel` (cálculo de calorias), DAOs (Room in-memory) e repositories (lógica de PRs), mas não chegaram a ser implementados
- **Testes instrumentados:** verificação do fluxo completo em emulador/dispositivo físico
- **Internacionalização:** todas as strings estão em `strings.xml`, o que facilita tradução futura
- **Funcionalidades pagas (Fase 2):** base de dados global de alimentos, estimativa calórica por IA
- **ID da aplicação:** ainda está como `com.example.baselift` (placeholder padrão do Android Studio), deveria ser atualizado para publicação